

# Type|Racer

---

## Documentation technique

Un jeu de vitesse de frappe embarqué dans Discord comme Activity.

# Table des matières

---

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>1 Identification du projet</b>                                | <b>2</b>  |
| <b>2 Le projet en chiffres</b>                                   | <b>3</b>  |
| <b>3 Description sommaire</b>                                    | <b>4</b>  |
| 3.1 Le produit                                                   | 4         |
| 3.2 Le contexte Discord                                          | 4         |
| 3.3 Le vocabulaire minimal                                       | 4         |
| <b>4 Fonctionnalités</b>                                         | <b>6</b>  |
| 4.1 Le Menu                                                      | 6         |
| 4.2 Solo : Practice                                              | 6         |
| 4.3 Solo : Apprendre                                             | 7         |
| 4.4 Multijoueur : entrer dans une Room                           | 7         |
| 4.5 Multijoueur : le salon d'attente                             | 8         |
| 4.6 Multijoueur : la course                                      | 8         |
| 4.7 Multijoueur : Floor is lava                                  | 9         |
| 4.8 Multijoueur : Spam                                           | 9         |
| 4.9 Multijoueur : le podium                                      | 10        |
| 4.10 Multijoueur : Play of the Game                              | 10        |
| 4.11 Paramètres                                                  | 10        |
| 4.12 Hors du jeu : ce que Discord montre                         | 11        |
| <b>5 Réalisation du projet</b>                                   | <b>12</b> |
| 5.1 Les trois axes d'une Race                                    | 12        |
| 5.2 Les trois Modes de jeu                                       | 12        |
| 5.3 La machine d'état d'une Race                                 | 13        |
| 5.4 Les Réglages de salon                                        | 13        |
| 5.5 Le multijoueur                                               | 14        |
| 5.6 Le cursus « Apprendre »                                      | 14        |
| 5.7 L'intégration Discord                                        | 15        |
| 5.8 Le mode debug dans l'iframe                                  | 16        |
| 5.9 La sécurité                                                  | 16        |
| 5.10 Le pipeline d'assets et la Rich Presence                    | 17        |
| 5.11 Le graphe de résultats, sans bibliothèque                   | 19        |
| 5.12 La chaîne d'intégration et le déploiement                   | 19        |
| 5.13 Méthode : le modèle de domaine fait autorité sur le nommage | 21        |
| 5.14 Méthode : la parité TypeScript ↔ Rust prouvée par vecteurs  | 21        |
| 5.15 Méthode : les décisions d'architecture consignées           | 22        |
| <b>6 Documentation technique</b>                                 | <b>23</b> |
| 6.1 Arborescence du projet                                       | 23        |
| 6.2 Architecture générale et frontière client/serveur            | 23        |
| 6.3 Le protocole WebSocket                                       | 23        |
| 6.4 Le contrat HTTP                                              | 23        |
| 6.5 Persistance                                                  | 23        |
| 6.6 Sécurité et frontière de confiance                           | 23        |
| 6.7 Tests et assurance qualité                                   | 23        |
| <b>7 Limites connues et suites</b>                               | <b>24</b> |
| <b>8 Annexes</b>                                                 | <b>25</b> |

# 1 Identification du projet

---

TypeRacerDiscord est un jeu de vitesse de frappe qui se joue **à l'intérieur de Discord**, comme Activity : une iframe lancée depuis un salon vocal, sans rien à installer et sans compte à créer. On y tape seul contre le chrono, ou à huit sur le même texte, avec trois règles de victoire différentes et un cursus de cent leçons pour apprendre à taper sans regarder ses doigts.

Le produit final est un binaire Linux unique, statique, qui sert à la fois l'interface et son API, publié en archive par la chaîne d'intégration à chaque version — plus les quelques assets déposés dans le portail développeur Discord.

Ce document décrit l'application telle qu'elle est à la version 0.0.4 : ce qu'elle fait pour un joueur, comment chacun de ses blocs a été construit, et comment le système tient debout une fois l'ensemble assemblé.

## 2 Le projet en chiffres

Un instantané, mesuré sur le dépôt à la version 0.0.4. Ces nombres ne sont pas un palmarès : ils donnent l'ordre de grandeur qu'il faut avoir en tête pour lire les sections suivantes — notamment le rapport entre le code de production et ce qui l'atteste.

| Grandeur                                               | Valeur    |
|--------------------------------------------------------|-----------|
| Lignes TypeScript (frontend/src, tests compris)        | 11 368    |
| Lignes Rust (backend/src, tests compris)               | 7 960     |
| Lignes de feuille de style (une seule)                 | 2 312     |
| Tests automatisés                                      | 382 + 159 |
| Langages tenus à parité, prouvée par vecteurs partagés | 2         |
| Décisions d'architecture consignées (ADR)              | 19        |
| Messages du protocole WebSocket                        | 29        |
| Leçons du cursus « Apprendre »                         | 100       |
| Migrations de base de données                          | 6         |
| Modes de jeu multijoueur                               | 3         |
| Issues fermées                                         | 121       |

Figure 1. – Le projet mesuré, version 0.0.4. Les 382 tests sont ceux de vitest côté TypeScript, les 159 ceux de cargo test côté Rust.

Deux chiffres méritent d'être lus ensemble. **Deux langages tenus à parité** signifie que le même algorithme de score existe en TypeScript et en Rust, et que des vecteurs de test communs interdisent aux deux versions de diverger — c'est le sujet de la section 5.14. **29 messages** est la taille du contrat entre le navigateur et le serveur pendant une course ; c'est aussi ce qui dérivera en premier si le protocole bouge, et ce que section 6.3 liste un par un.

## 3 Description sommaire

---

### 3.1 Le produit

Un joueur ouvre Discord, entre dans un salon vocal, clique sur la fusée et lance TypeRacer. L'application s'affiche dans une iframe, à l'intérieur du client Discord — pas dans un onglet, pas dans une fenêtre à côté. Elle reconnaît le joueur toute seule : son nom et son avatar viennent de la session Discord déjà ouverte, il n'y a rien à saisir.

De là, deux chemins. **Solo** : un texte apparaît, le chrono part à la première frappe, et l'écran de résultats affiche vitesse, précision et une courbe seconde par seconde. **Multijoueur** : les joueurs du même salon se retrouvent automatiquement dans un salon d'attente, l'hôte règle la partie, et tout le monde tape le même texte pendant que des voitures avancent sur une piste. Un troisième chemin, **Apprendre**, est un cursus de cent leçons qui ne se joue pas contre les autres mais contre ses propres doigts.

Pendant tout ce temps, les autres membres du salon vocal voient dans Discord ce que le joueur est en train de faire — « En course », « S'entraîne », avec le visuel du mode choisi. C'est la Rich Presence, et elle fonctionne même pour ceux qui n'ont pas ouvert l'Activity.

### 3.2 Le contexte Discord

Une Activity n'est pas un site web hébergé quelque part que Discord irait ouvrir. C'est une page servie **à travers Discord**, dans une iframe verrouillée, sous une politique de sécurité de contenu qui refuse toute requête réseau ne passant pas par un préfixe imposé. Cette contrainte-là a façonné plus de code que n'importe quel choix d'architecture — elle revient à la section 5.7.

Trois conséquences structurent tout le reste :

- **L'origine est unique.** Discord ne redirige qu'une seule adresse vers le projet. Le serveur Rust sert donc à la fois le build du frontend et l'API.
- **Il n'y a pas d'URL.** L'adresse de l'iframe est figée par Discord ; la navigation se fait entièrement par boutons, écran par écran.
- **Il n'y a pas de console.** Un développeur qui déboguait dans le client Discord ne voit rien du tout — d'où un bandeau d'erreurs affiché dans le jeu lui-même (section 5.8).

### 3.3 Le vocabulaire minimal

Le projet tient un glossaire de domaine complet — une quarantaine de termes, avec pour chacun les mots qu'il est interdit d'employer à sa place — dans `CONTEXT.md`, à la racine du dépôt. Ce document n'en reprend pas la lettre. Voici les douze termes sans lesquels les sections suivantes ne se lisent pas ; les autres sont introduits là où ils servent.

| Terme                         | Ce qu'il désigne                                                                                                                                           |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Run</b>                    | Une tentative de frappe, du premier caractère à la fin. L'unité à laquelle tout le reste s'accroche. Deux espèces : Practice et Race.                      |
| <b>Practice</b>               | Un Run solo en saisie libre : le retour arrière est permis, une faute peut rester non corrigée, on peut taper au-delà du mot.                              |
| <b>Race</b>                   | Un Run compétitif dans une Room, sur la même saisie libre — mais qui ne se termine qu'une fois le texte entier tapé <b>exactement</b> .                    |
| <b>Room</b>                   | La session multijoueur qui réunit les joueurs sur un même texte. Identifiée par une clé unique prenant deux formes : un salon vocal, ou un Code de partie. |
| <b>Mode</b>                   | La règle solo qui décide du texte présenté et de la fin du Run : Time, Words, Quotes ou Zen. Exactement un par Run — et <b>aucun</b> en Race.              |
| <b>Mode de jeu</b>            | La règle qui décide comment une Race se <b>gagne</b> : Normal, Floor is lava ou Spam. Un axe à part entière, jamais cumulable.                             |
| <b>Source de texte</b>        | D'où vient le texte d'une Race — une citation, ou des mots générés. Elle décide du texte, jamais de la mesure.                                             |
| <b>Réglage de salon</b>       | Une option de Room posée par l'hôte hors course et imposée à tout le monde : durée du décompte, taille max, Difficulté, Mode de jeu...                     |
| <b>Preference</b>             | Ce qu'un joueur veut voir <b>sur sa machine à lui</b> : police, palette, pseudo affiché. Ne quitte jamais l'appareil et n'influence aucun score.           |
| <b>Keystroke log</b>          | La chronologie brute des frappes d'un Run : quoi, et à quel instant. La matière première dont tout le reste est dérivé.                                    |
| <b>Scoreboard autoritaire</b> | Les chiffres qui font foi, recalculés par le serveur Rust depuis le Keystroke log à la fin d'un Run. En multijoueur, c'est aussi l'anti-triche.            |
| <b>Gap (écart)</b>            | De combien de secondes un joueur a fini derrière le vainqueur. C'est le chiffre qu'on dit à voix haute à l'arrivée, pas la vitesse absolue.                |

Figure 2. – Les douze termes de travail. Le glossaire complet, avec ses interdits de vocabulaire, reste dans CONTEXT.md.

### EN CLAIR Le « recompute autoritaire »

Le navigateur du joueur affiche un compteur de vitesse pendant qu'il tape, mais ce chiffre-là n'est **jamais** celui qu'on enregistre. À la fin, le navigateur envoie au serveur la liste brute de ses frappes — la touche et l'instant, rien d'autre. Le serveur **rejoue** cette liste contre le texte qu'il possède, et recalcule tout lui-même. C'est un peu comme rendre sa copie d'examen : ce n'est pas la note que l'élève s'est donnée qui compte, c'est celle du correcteur. Le joueur peut mentir sur sa vitesse, il ne peut pas mentir sur ses frappes sans réécrire une partie entière cohérente.

## 4 Fonctionnalités

---

Cette section décrit le produit **à l'œil du joueur** : ce qu'il voit, dans l'ordre où il le rencontre. Aucun nom de fichier, aucun détail d'implémentation — ceux-là viennent à la section 5 et à la section 6.

### 4.1 Le Menu

Le point d'arrivée. Quatre portes : Solo, Multijoueur, Paramètres, Quitter. « Quitter » ferme réellement l'Activity dans Discord, et disparaît quand le jeu tourne hors Discord.



Figure 1. – Le Menu, écran d'arrivée. Les quatre entrées, le wordmark, et l'accès au guide « Comment jouer » — affiché d'office la toute première fois.

### 4.2 Solo : Practice

Une barre de configuration au-dessus de la zone de frappe : le Mode (Time / Words / Quotes / Zen), sa valeur, puis les modificateurs de texte cumulables — ponctuation, chiffres. Deux Modes particuliers escamotent ces contrôles parce qu'ils ne s'y appliquent pas : Quotes (la longueur appartient à la citation) et Zen (il n'y a pas de texte cible du tout).



Figure 2. – La barre de configuration solo dépliée. Chaque contrôle n'apparaît que lorsqu'il a un sens : choisir « citation » fait disparaître la longueur et les modificateurs.

Le chrono ne part pas au clic mais à la **première frappe** : rien n'est masqué, personne n'attend, il n'y a pas de décompte. Le texte défile par fenêtre de trois lignes, le mot en cours restant toujours sur celle du milieu.



Figure 3. – Un Practice en cours. Le mot actif reste sur la ligne du milieu ; les caractères déjà tapés changent de couleur, une faute passe en rouge.

À la fin, l'écran de résultats affiche le scoreboard recalculé par le serveur — vitesse nette, vitesse brute, précision, décompte des caractères — et la courbe seconde par seconde. Un bouton rejoue le Run à sa vitesse réelle, fautes comprises.

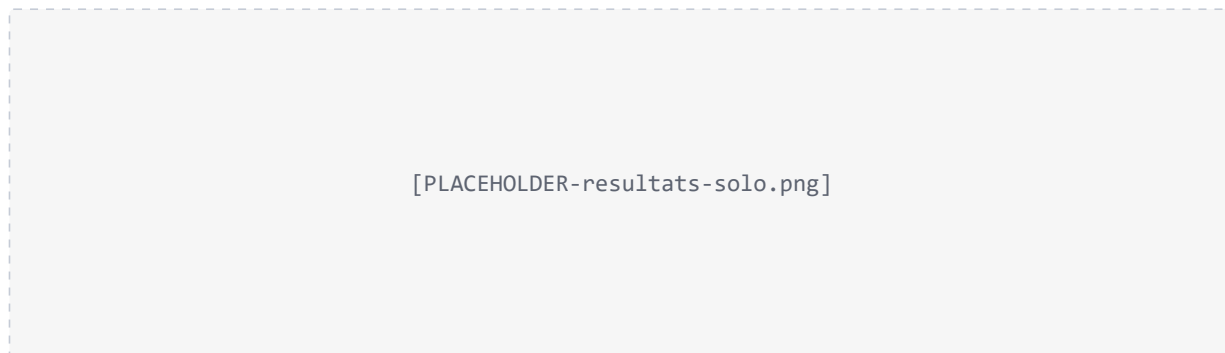


Figure 4. – L'écran de résultats. Le graphe seconde par seconde est un SVG construit à la main : la dépendance à une bibliothèque de graphiques a été retirée du projet, section 5.11).

### 4.3 Solo : Apprendre

Un cursus de cent leçons, débloquées une à une. Chacune enseigne un point précis — la position des mains, la rangée de repos, les majuscules, la ponctuation, les chiffres, puis les vrais mots — et se termine par un exercice. Le seul critère pour passer à la suivante est la **précision**, jamais la vitesse, et la barre monte par paliers au fil du cursus.



Figure 5. – La liste des leçons. Trois états visibles : verrouillée, disponible, terminée. La progression suit le joueur, pas l'appareil.

### 4.4 Multijoueur : entrer dans une Room

Trois portes, présentées ensemble sur le même écran :



**Le salon** — rejoindre la Room du salon vocal courant. Elle est créée à la volée si elle n'existe pas — la clé vient de Discord, elle ne peut pas être mal tapée.

**Créer** — ouvrir une Room neuve et recevoir un Code de partie de cinq caractères, lisible à voix haute.

**Rejoindre par code** — saisir un code. Celui-ci ne crée jamais rien : un code inconnu répond « introuvable » et laisse corriger sur place.

Le code est ce qui permet à des joueurs de **serveurs Discord différents** de courir ensemble.

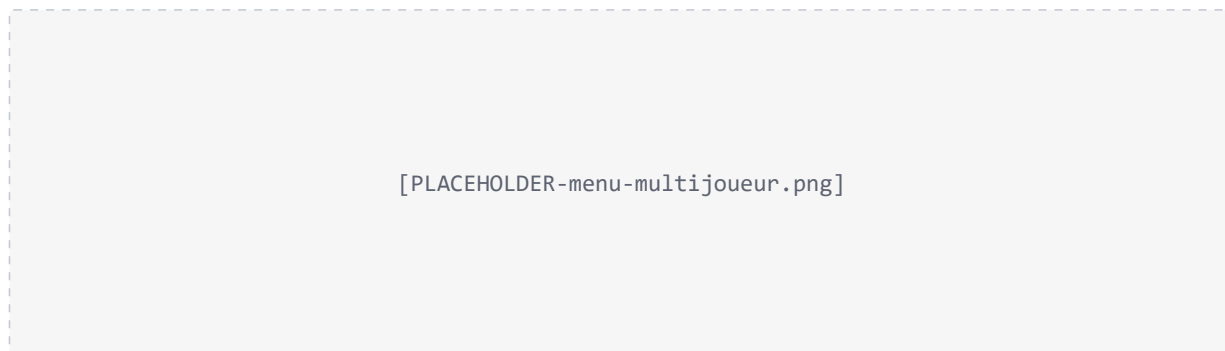


Figure 6. – Les trois portes d'entrée d'une Room, avec le champ de saisie du code au même endroit : un code refusé ramène le joueur là où il peut le corriger.

## 4.5 Multijoueur : le salon d'attente

Trois colonnes. À gauche les présents, avec avatar, pseudo et couronne pour l'hôte. Au centre les réglages de la partie. À droite, dans un cadre, le visuel du Mode de jeu choisi et ce qu'il implique.

Ce que l'hôte règle : le Mode de jeu, la Source du texte et sa longueur, la durée du décompte, la taille maximale de la Room, la Difficulté, le ready-check — et, selon le Mode de jeu, l'intervalle d'élimination ou le mot et les seuils. Tout le monde voit ces réglages : ils s'imposent à tous, pas seulement à celui qui les choisit.

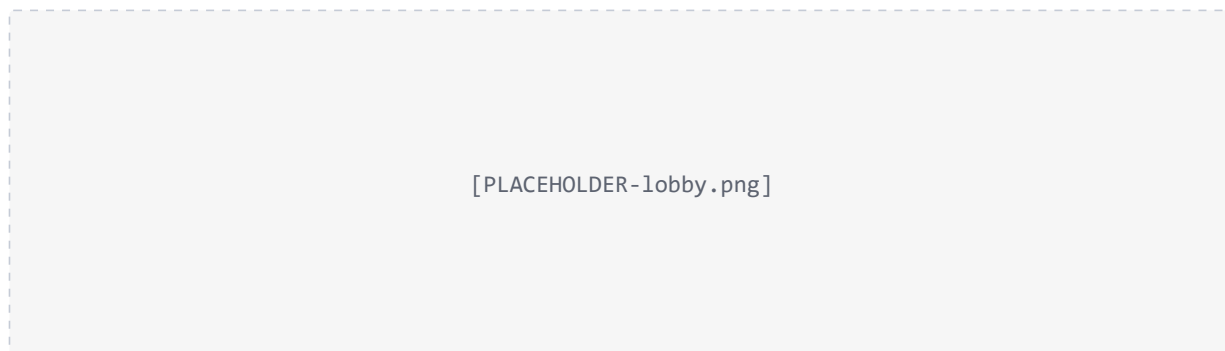


Figure 7. – Le salon en trois colonnes. Le Code de partie est masqué par défaut et se révèle d'un clic — on ne diffuse pas par accident l'entrée de sa partie.

## 4.6 Multijoueur : la course

Une ligne par joueur : l'avatar en tête de progression, le pseudo, la vitesse en direct à hauteur de la ligne d'arrivée. Le décompte dure ce que l'hôte a réglé, texte entier visible pendant l'attente — il n'y a rien à masquer, tout le monde part au même instant.



Figure 8. – La piste pendant une course classique. La position de chaque voiture suit le nombre de caractères corrects, pas le temps.

#### 4.7 Multijoueur : Floor is lava

À intervalle régulier, le joueur le moins avancé brûle. La course s'arrête à l'instant où il ne reste qu'un vivant : le survivant gagne **sans avoir terminé le texte**. C'est la seule course qui se finit sans que personne ne franchisse de ligne d'arrivée — le mode impose d'ailleurs un texte trop long pour être achevé. On ne gagne pas en tapant vite, on gagne en n'étant jamais dernier.



Figure 9. – Floor is lava en action. Ce qui compte n'est pas la position absolue mais la position **relative** : le dernier au moment du tic disparaît.

#### 4.8 Multijoueur : Spam

Un seul mot, répété indéfiniment. L'hôte choisit le mot — dans la liste existante ou en le tapant lui-même — un seuil de répétitions, et un plafond de temps. La course s'arrête dès que l'un des deux tombe : soit quelqu'un verrouille le seuil, soit le temps expire et c'est le plus grand nombre de répétitions correctes qui l'emporte.



Figure 10. – Le mode Spam et son compteur de répétitions, qui remplace la vitesse à la ligne d'arrivée : c'est la grandeur qui décide de la victoire.

## 4.9 Multijoueur : le podium

Trois marches, les autres joueurs visibles à côté, et le **Gap** en gros caractères — l'écart au vainqueur en secondes. Cliquer sur un joueur déplie sa courbe seconde par seconde, sans aucune requête réseau : tout est arrivé avec le message de fin de course.

Les deux Modes de jeu n'ont pas de Gap, puisque personne ne franchit de ligne. Chacun affiche à sa place ce qui décide vraiment de son classement : le temps de survie pour Floor is lava, le nombre de répétitions pour Spam.

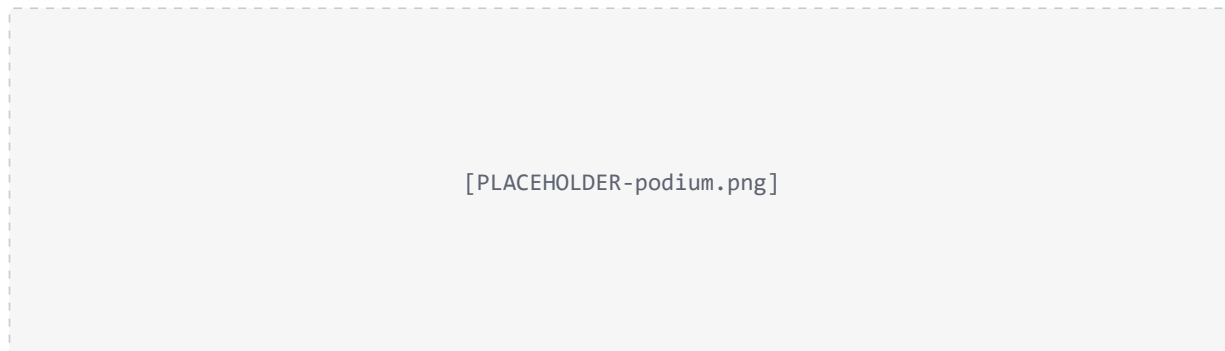


Figure 11. – Le podium, Gap en tête d’affiche. Les abandons et les échecs figurent derrière tous les finisseurs, jamais mêlés à eux.

## 4.10 Multijoueur : Play of the Game

Quand deux joueurs ont fini à moins de deux secondes l'un de l'autre — où qu'ils soient au classement — leurs dernières secondes sont rejouées côte à côte, au ralenti, sur une **horloge partagée**. C'est cette horloge unique qui en fait un duel plutôt que deux relectures posées l'une à côté de l'autre. Sans photo-finish, il n'y a pas de Play of the Game du tout : le bouton n'apparaît pas.

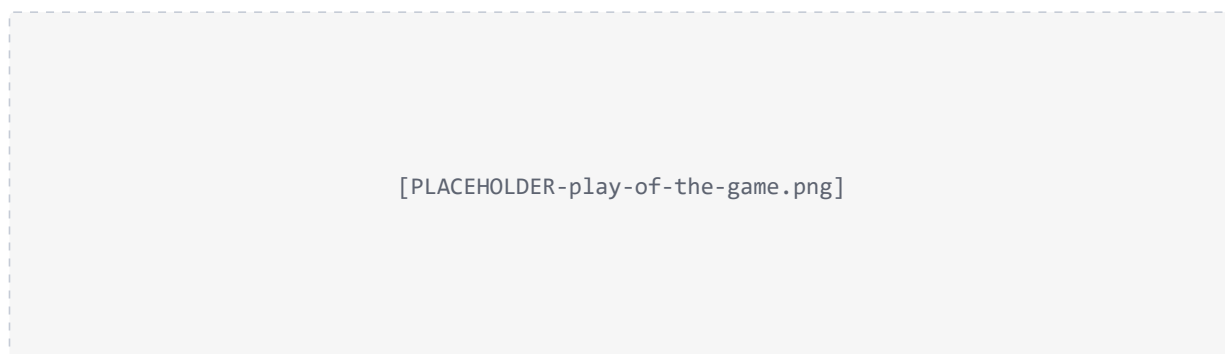


Figure 12. – Play of the Game : deux Keystroke logs rejoués sur la même horloge, sur les dernières secondes avant la première des deux arrivées.

## 4.11 Paramètres

Une ligne par réglage : libellé et explication à gauche, contrôle à droite, groupés en sections — Apparence, Solo, Son, et une zone à risque pour exporter, importer ou effacer ses données. Le choix de la police de frappe s'affiche directement dans la police concernée : l'aperçu **est** le contrôle.

Ces réglages ne quittent jamais l'appareil. Deux joueurs de la même course peuvent voir des polices et des couleurs différentes tout en tapant exactement le même texte, et changer un réglage n'invalide jamais un record.



Figure 13. – L'écran Paramètres. Chaque type de contrôle — segmenté, curseur, bascule, nombre — est né avec le premier réglage qui en avait besoin.

## 4.12 Hors du jeu : ce que Discord montre

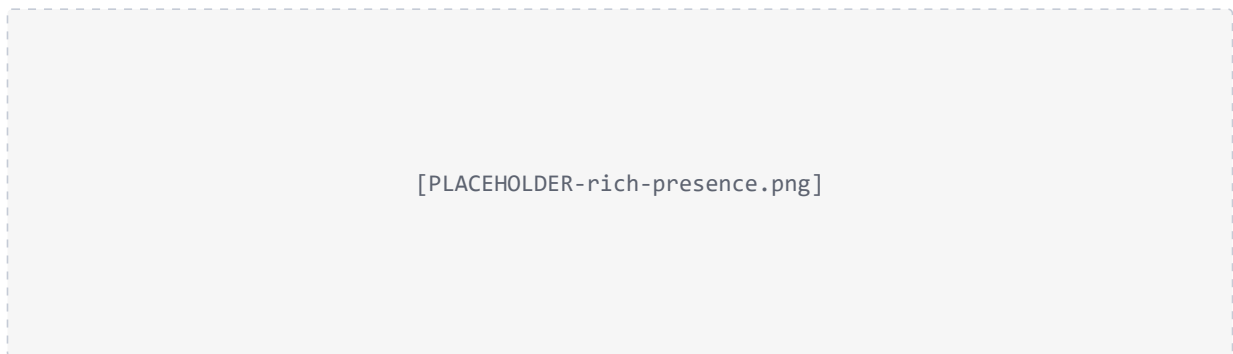


Figure 14. – La Rich Presence telle qu'un autre membre du salon la voit, sans avoir ouvert l'Activity : l'état courant, le visuel du Mode de jeu, le compte des présents et un chrono. C'est la seule preuve visible que le câblage fonctionne de bout en bout.

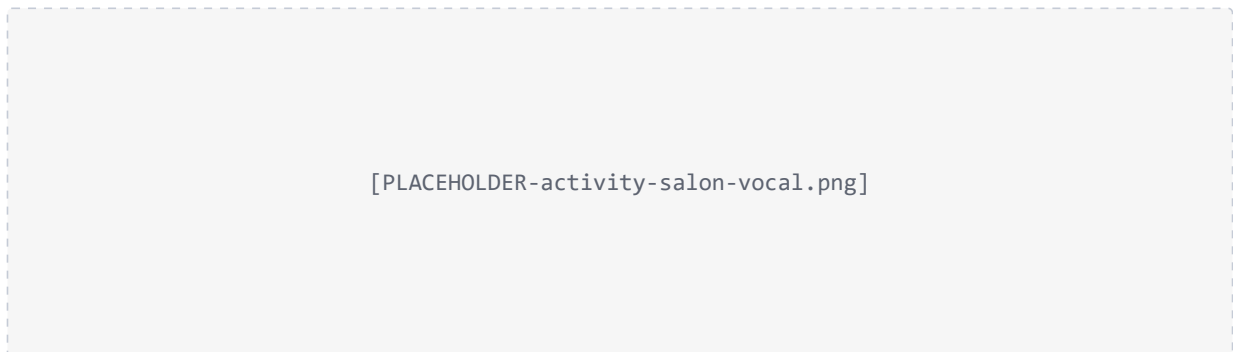


Figure 15. – L'Activity lancée depuis un salon vocal, dans le client Discord. L'iframe occupe la zone de conversation ; le jeu s'échelonne à la fenêtre plutôt que de faire défiler.

## 5 Réalisation du projet

Cette section change de point de vue : on ne regarde plus l'écran, on regarde le chantier. Un paragraphe descriptif par bloc construit, dans l'ordre où ils s'empilent. Les encadrés « récit » n'interrompent le catalogue que pour les problèmes qui ont **réellement** résisté — un chantier qui s'est bien passé n'a pas d'histoire à raconter, et un récit posé partout ne raconterait plus rien.

Les trois derniers chantiers ne produisent aucune fonctionnalité : ce sont des chantiers de **méthode**, et ils expliquent pourquoi les précédents ont pu être menés sans se marcher dessus.

### 5.1 Les trois axes d'une Race

Le concept le plus facile à confondre du domaine, et celui qu'il a fallu séparer en premier. Trois réglages agissent sur une même course, et **aucun** n'est une variante des deux autres.

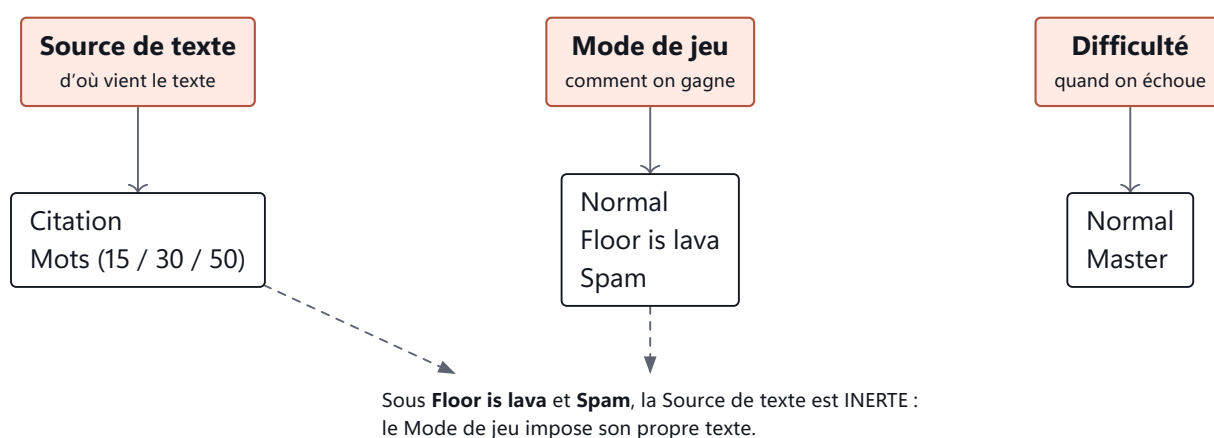


Figure 16. – Les trois axes d'une Race. La Source décide du texte et jamais de la mesure ; le Mode de jeu décide de la victoire ; la Difficulté est une condition d'échec **individuelle**, évaluée sur le seul journal de frappe du joueur concerné et jamais contre les autres.

Une Race n'a délibérément **pas** de Mode : sa règle de fin est toujours « le texte entier, exactement ». Un Mode Time en course impliquerait des voitures sans ligne d'arrivée commune, et Zen n'a pas de fin du tout. Ce que l'hôte choisit, c'est la provenance du texte — le recalcul autoritaire, lui, mesure toujours la même chose.

### 5.2 Les trois Modes de jeu

Normal est le comportement d'origine : le premier à taper tout le texte, exactement. Les deux autres ont été ajoutés comme des **axes**, pas comme des options, précisément pour qu'ils ne se cumulent jamais entre eux.

**Floor is lava** remplace la victoire par la survie. Un métronome serveur bat à l'intervalle réglé ; à chaque battement, le moins avancé brûle. Le classement est l'ordre des morts, inversé. Deux conséquences ont dû être tirées jusqu'au bout : le mode impose son propre texte, assez long pour que personne ne l'achève — un mode qui se gagne en survivant ne doit pas offrir de porte de sortie par l'arrivée — et un joueur brûlé porte tout de même un **vrai score partiel**, recalculé sur ce qu'il a eu le temps de taper. Ce score ne le classe jamais ; il sert à l'afficher et à choisir le duel d'après-course.

**Spam** remplace le texte par un mot unique, diffusé sans fin. Deux réglages décident de l'arrêt — un seuil de répétitions et un plafond de temps — et le premier des deux qui tombe arrête tout le monde. Le compte de répétitions annoncé par un client est déclaratif : il peut **arrêter** la course, il ne peut pas la

gagner. Le classement vient du recompte serveur, qui rejoue chaque journal de frappe contre sa propre copie du mot.

#### RÉCIT Floor is lava éliminait pendant le décompte

**Problème** Le métronome démarrait avec la course, pas avec la fin du décompte. Pendant les cinq secondes d'attente, personne n'a encore tapé le moindre caractère : tout le monde est donc à égalité au dernier rang. Le premier battement éliminait alors **tous** les joueurs d'un coup, et la Room se figeait dans un état sans vivant et sans vainqueur.

**Décision** Faire partir le métronome au signal de départ partagé et non à la création de la course, et traiter l'égalité générale comme un cas nommé plutôt que comme un tri qui « tombe bien » la plupart du temps.

**Résultat** Le mode a servi d'avertissement pour les suivants : toute règle comparative doit dire ce qu'elle fait quand **tous** les joueurs sont à égalité, y compris à zéro. Le cas dégénéré n'est pas un cas rare, c'est l'état initial de chaque course (issue 159).

#### EN CLAIR Un texte « généré à partir d'une graine »

Le texte d'une course n'est pas tiré au hasard puis envoyé à tout le monde : on tire un **nombre**, la graine, et le texte se déduit de ce nombre par un calcul sans surprise. Même graine, même texte, toujours, sur n'importe quelle machine. C'est ce qui permet au serveur Rust et au navigateur de fabriquer le même texte chacun de son côté sans se le transmettre — et c'est aussi ce qui rend les tests possibles : une graine fixée donne un texte connu d'avance.

### 5.3 La machine d'état d'une Race

Chaque course traverse les mêmes phases, et chaque joueur en sort par l'une de cinq portes. Ces cinq sorties ne sont pas des nuances de la même chose : elles se distinguent par **qui décide**.

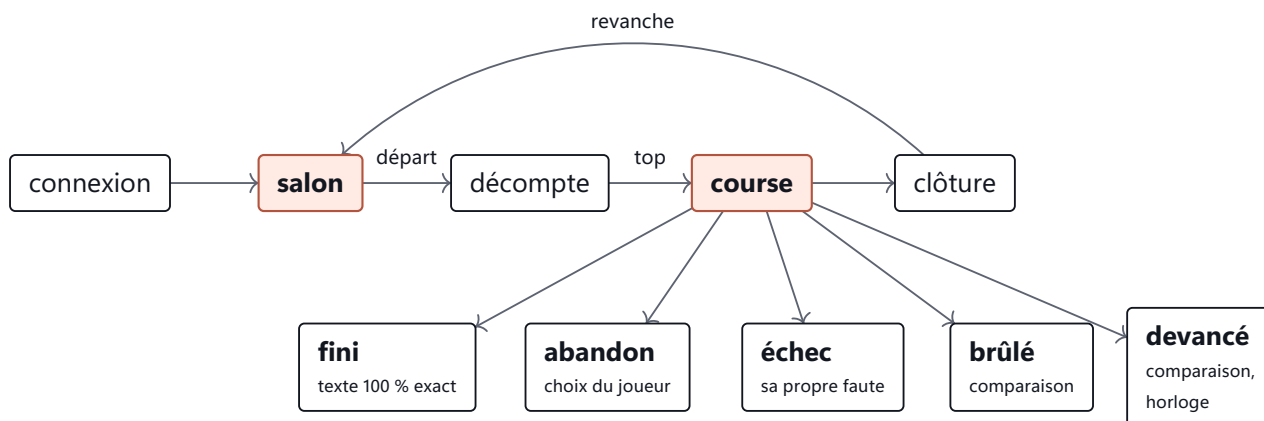


Figure 17. – Les phases d'une Race et les cinq états terminaux d'un partant. **Fini** est la seule sortie par la ligne d'arrivée. **Abandon** vient du joueur lui-même — et une déconnexion produit exactement le même enregistrement. **Échec** vient de sa propre faute sous Difficulté Master. **Brûlé** et **devancé** ne viennent ni d'un choix ni d'une faute : ils viennent d'être comparé aux autres.

Un état terminal ne se reprend pas : une fois posé, il ne peut plus être écrasé par un message arrivé en retard. Un abandon garde par ailleurs la position qu'avait le joueur — sa voiture s'arrête là où il s'est arrêté, elle ne retombe pas à zéro.

### 5.4 Les Réglages de salon

Sept réglages, plus ceux propres à chaque Mode de jeu. Ils partagent tous la même frontière : posés par l'hôte seul, acceptés hors course seulement, et rediffusés à tout le salon dès qu'ils sont acceptés. Un réglage refusé est ignoré en silence côté serveur — le client n'est jamais en position d'imposer quoi que ce soit.

Le point délicat n'était pas la liste mais la **bascule**. Passer d'un Mode de jeu à un autre régénère le texte, puisque chaque mode impose le sien. Or les réglages préparés pour un mode qu'on quitte doivent survivre : celui qui règle Spam, passe voir Floor is lava, puis revient, doit retrouver son mot et ses seuils tels qu'il les avait laissés. Les réglages vivent donc à plat sur la Room plutôt que dans la variante du Mode de jeu — un écart assumé par rapport à la décision d'architecture qui les décrivait, et documenté comme tel dans le code.

## 5.5 Le multijoueur

Une Room est identifiée par une clé unique qui prend deux formes : un salon vocal Discord, ou un Code de partie de cinq caractères. Une seule table, deux formes — et trois portes d'entrée aux droits différents, décrites à la section 6.3. L'alphabet du code exclut les caractères qui se confondent à l'oral et à l'œil : ni 0/0, ni 1/I/L. Un code n'est jamais réservé ni persisté ; il vit tant que sa Room vit, et meurt avec elle.

Le premier arrivé est l'hôte, et le rôle passe au suivant s'il part. La Room plafonne à huit présents. Les partants sont **figés au départ** : quelqu'un qui rejoint pendant une course occupe une place mais n'est pas dans cette course-là.

À l'arrivée, le message de fin porte le classement complet de tous les partants, avec leur courbe seconde par seconde — pas seulement l'ordre. C'est ce qui permet au podium d'afficher le Gap et de déplier le graphe de n'importe quel joueur sans un seul aller-retour réseau. Ce choix a une raison précise : la composition d'une course vit en mémoire et meurt avec la Room, donc le serveur ne pourrait pas vérifier après coup qu'un demandeur en faisait partie. Sur le WebSocket, la question ne se pose pas — l'autorisation **est** la connexion.

### RÉCIT Le verrou des Rooms et l'aller-retour réseau

**Problème** Une Room dont la Source est « citation » doit aller chercher son texte sur une API externe. Mais l'état des Rooms est protégé par un verrou classique, celui qui ne peut pas être tenu à travers une attente réseau : le tenir pendant la requête bloquerait toutes les autres Rooms du serveur pendant la latence de l'API, et le compilateur Rust refuse d'ailleurs la construction.

**Décision** Découper en trois temps : lire la Source sous verrou, **relâcher**, chercher le texte sans verrou, reposer le résultat sous verrou et rediffuser l'état. Et faire naître toute Room avec un texte de mots déjà en place, même quand sa Source est « citation ».

**Résultat** Une Room est jouable immédiatement, sans réseau. Le salon voit l'ancien texte puis le nouveau. Une course lancée entre-temps annule la pose du texte en vol, devenu périmé. Le repli après échec de l'API n'est pas un état à part — la Room bascule **réellement** sur des mots générés, et c'est ce qu'elle annonce.

### EN CLAIR Pourquoi un verrou ne traverse pas une attente

Un verrou sert à garantir qu'une seule partie du programme touche une donnée à la fois. Le tenir, c'est faire attendre tous les autres. Tant qu'on ne fait que quelques calculs, l'attente se compte en microsecondes. Mais si on garde le verrou pendant qu'on interroge un serveur à l'autre bout d'Internet, on fait attendre tout le monde pendant des centaines de millisecondes — pour une requête qui ne concernait qu'une seule Room. D'où le découpage en trois temps : on ne tient jamais la porte fermée pendant qu'on va faire les courses.

## 5.6 Le cursus « Apprendre »

Cent leçons, du placement des mains à la fluidité, en passant par les rangées, les majuscules, la ponctuation et les chiffres. Le contenu est une **donnée** — un fichier de description séparé du moteur — précisément pour qu'ajouter ou réécrire une leçon ne demande pas de toucher au code.

Deux décisions structurent le cursus. La première : le seul critère de déverrouillage est la précision, jamais la vitesse, avec un barème par tranches qui se durcit au fil des leçons et se modifie en un seul endroit.

La seconde : un exercice de leçon **n'est pas un Run**. Il n'entre pas dans l'historique, ne produit jamais de record, et n'est jamais soumis au serveur pour recalcul. La progression, elle, suit le joueur et non l'appareil — le serveur en conserve toujours le maximum atteint, jamais la dernière valeur reçue.

## 5.7 L'intégration Discord

L'identité du joueur ne vient d'aucun formulaire : elle vient de la session Discord déjà ouverte, par un enchaînement en quatre temps entre l'iframe, le client Discord et le serveur.

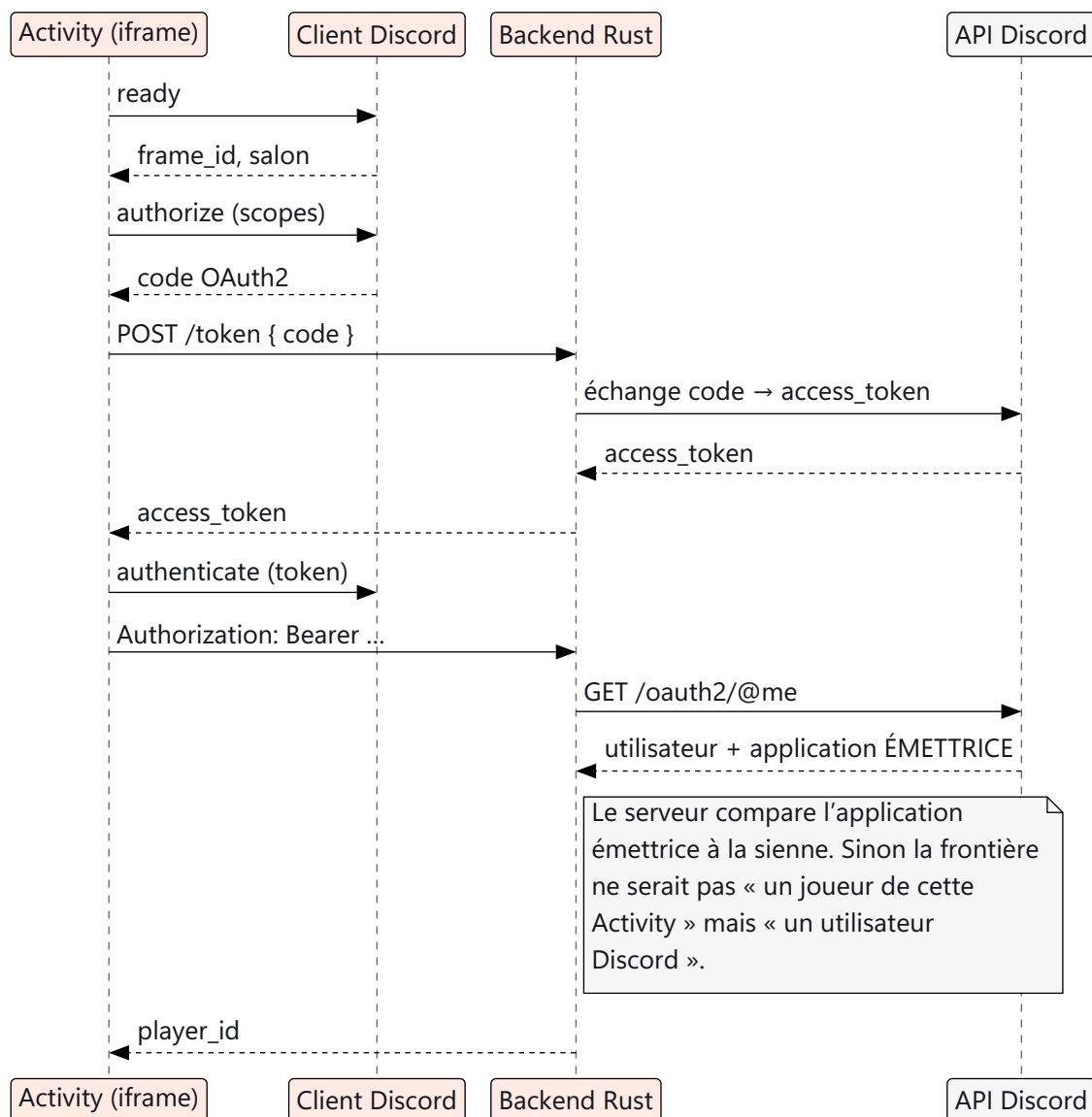


Figure 18. – Le handshake d'identité. Le secret client ne quitte jamais le serveur : c'est lui, et non l'iframe, qui échange le code contre un jeton. La dernière vérification est celle qui compte — un jeton d'accès Discord est valable sur l'endpoint d'identité **quelle que soit l'application qui l'a obtenu** (issue 150).

### EN CLAIR OAuth2, ou « prouver son identité sans donner son mot de passe »

Le jeu ne connaît jamais le mot de passe Discord de personne. Discord remet à l'application un **jeton** temporaire, qui dit « ce porteur est bien tel utilisateur, et il a accepté de partager son nom ». Le jeu montre ce jeton au serveur, qui va demander à Discord à qui il appartient. Le piège que le projet a dû traiter : un jeton obtenu par **une autre** application Discord répond aussi à cette question. Il faut donc demander en plus qui l'a émis, sinon n'importe quel utilisateur de Discord pourrait se présenter comme joueur.



Le reste de l'intégration tient dans deux contraintes. La première est la politique de sécurité de l'iframe, sujet du récit ci-dessous. La seconde est qu'une Activity non publiée reste **invisible** tant qu'un testeur n'a pas accepté son invitation par courriel — un piège de portail qui n'a rien à voir avec le code, et qui a coûté une session entière avant d'être identifié.

#### RÉCIT La politique de sécurité de l'iframe Discord

**Problème** Dans l'iframe d'une Activity, **toute** requête réseau qui ne passe pas par un préfixe imposé est refusée. Le document et les scripts se chargent normalement : l'interface s'affiche, parfaitement intacte, et rien ne fonctionne. Aucune erreur visible, aucun message — et la console du navigateur n'existe pas dans le client Discord.

**Décision** Une seule fonction décide du préfixe, en observant si l'application tourne dans une iframe Discord ou non, et les quatre points d'accès réseau du projet passent tous par elle. Aucun appel direct nulle part.

**Résultat** Le proxy Discord retire le préfixe **avant** d'appliquer sa redirection : le serveur et le développement local n'ont donc rien à savoir de tout ça. Ce symptôme — « l'interface va bien, le réseau est mort » — est devenu le premier réflexe de diagnostic du projet, et il a directement motivé le bandeau d'erreurs de la section suivante.

#### EN CLAIR La politique de sécurité de contenu d'une iframe

Une page web peut déclarer une liste de ce que le navigateur a le droit de charger et de contacter. Discord en impose une aux Activities, et elle est très serrée : rien ne sort sans passer par un chemin balisé qu'elle contrôle. L'intention est saine — une Activity malveillante ne peut pas discrètement envoyer les données de ses joueurs ailleurs. L'effet secondaire est cruel : une requête refusée ne casse pas la page, elle échoue silencieusement.

## 5.8 Le mode debug dans l'iframe

Corollaire direct du récit précédent : sans console, une erreur JavaScript dans Discord est totalement invisible. Le jeu attrape donc lui-même les erreurs non rattrapées et les promesses rejetées, et les affiche dans un bandeau rouge en bas de l'écran, refermable d'un clic. C'est une dizaine de lignes, et c'est ce qui a rendu le débogage dans Discord possible.



Figure 19. – Le bandeau d'erreurs. Il n'existe que parce que la console n'existe pas : dans le client Discord, c'est le seul canal de diagnostic disponible.

Pour une vraie console, il reste un chemin : ouvrir Discord **au navigateur** et lancer l'activité de là. Les outils de développement sont alors ceux du navigateur, sur l'iframe.

## 5.9 La sécurité

La passe de sécurité a porté sur cinq fronts distincts, tous menés après que l'application ait été jouable — et c'est précisément ce qui a permis de les traiter comme des frontières plutôt que comme des rustines.

**La frontière d'identité** — le jeton doit avoir été émis pour **cette** application, pas seulement être un jeton Discord valide.

**Les plafonds de requêtes** — posés dans l'extracteur d'identité lui-même, donc un endpoint authentifié ajouté demain est couvert sans qu'on écrive une ligne. Le proxy de citations en a un second, plus serré,

parce qu'il consomme un quota mensuel partagé. Un dépassement répond explicitement, il ne coupe pas la connexion en silence.

**Les en-têtes de réponse** — politique de sécurité de contenu, refus du reniflage de type, pas de référent. Avec une exception assumée et nommée — l'encadrement par Discord **doit** rester autorisé, l'Activity n'étant qu'une iframe ; durcir ce point-là rendrait le jeu invisible.

**Les entrées venues du réseau** — le pseudo est tronqué, le hash d'avatar validé et jeté s'il ne correspond pas au format attendu. On ne transporte jamais d'URL d'avatar — une URL fournie par un client serait une adresse arbitraire chargée dans le navigateur des sept autres.

**La frontière de confiance de la course** — le sujet du récit ci-dessous, et de la section 6.6.

#### RÉCIT Un « j'ai fini » partiel gagnait la course

**Problème** Le message de fin de course était cru sur parole. Trois caractères tapés très vite suffisaient donc à annoncer une arrivée : la durée était minuscule, la vitesse calculée absurde — de l'ordre de 800 mots par minute — et le joueur prenait la première place.

**Décision** Rejouer le journal de frappe côté serveur contre **son** texte et vérifier qu'il atteint réellement la fin, avant d'enregistrer quoi que ce soit. Une fin non confirmée n'est pas rejetée en silence : elle est enregistrée comme un abandon, le seul verdict à la fois vrai et débloquent pour les autres.

**Résultat** Ce n'était pas un correctif isolé mais l'entrée d'une frontière de confiance entière à reconstruire, poursuivie sur plusieurs issues : quels champs le client a-t-il le droit d'affirmer, lesquels ne font qu'**arrêter** quelque chose, et lesquels le serveur possède seul. La règle qui en est sortie tient en une phrase — une déclaration du client peut arrêter une course, jamais la gagner (issues 160, 163, 164, puis 186).

## 5.10 Le pipeline d'assets et la Rich Presence

Les assets visuels du projet ne sont pas des images dessinées dans un éditeur : ce sont des **programmes**. Une bibliothèque de composants vectoriels — la voiture, le clavier, le sol de lave, les feux de départ, le wordmark — est écrite en Typst, et chaque asset est un petit fichier qui l'importe, pose sa page et compose sa scène. Un script régénère l'ensemble d'une commande.

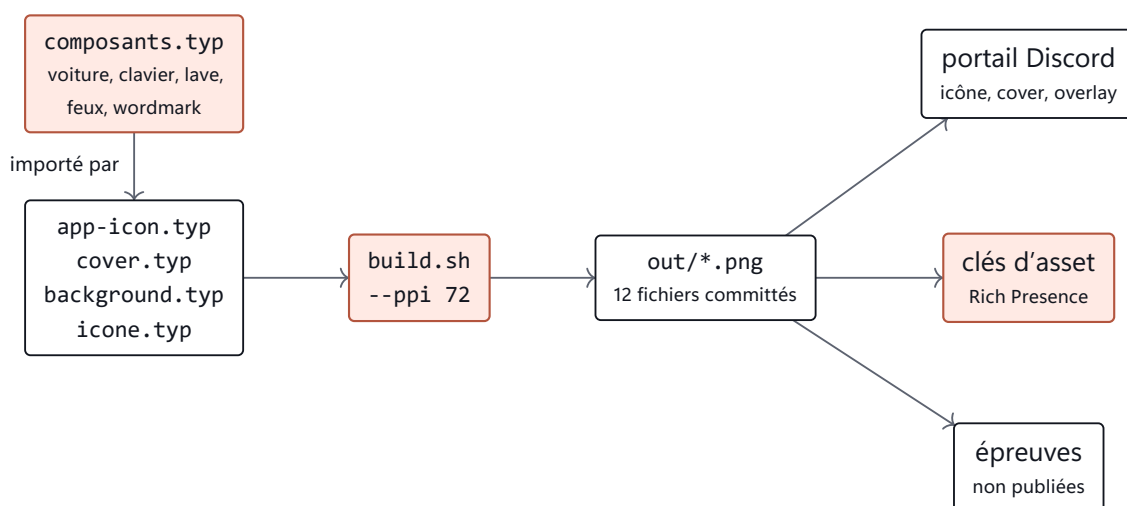


Figure 20. – Le pipeline d'assets. Le point à retenir est à droite : **le nom du fichier EST la clé d'asset** que le code demande à Discord. Renommer un PNG ne casse rien à la compilation — l'état correspondant perd simplement son visuel, silencieusement.

Deux règles ont été posées d'emblée, et elles se justifient l'une l'autre. La première : la page est déclarée en points et l'export forcé à 72 pixels par pouce, ce qui fait qu'un point vaut exactement un pixel — sans quoi le réglage par défaut livre le double des dimensions demandées, ce qui ne se remarque qu'au téléversement. La seconde : le générateur pseudo-aléatoire qui dessine la croûte de basalte du sol de

l'ave est **déterministe**. Deux exécutions du script produisent des PNG identiques au bit près ; sans cela, chaque régénération salirait le diff des images committées.



Figure 21. – La voiture, composant central. Une seule silhouette fermée plutôt qu'un assemblage de rectangles : à 48 pixels dans l'étagère Discord, il ne reste que le contour, et un contour unique y survit là où une pile de formes se brouille.



Figure 22. – L'épreuve de contrôle, agrandie ici pour être lisible : elle ne fait que 48 pixels de côté. C'est à cette taille que l'icône apparaît dans l'étagère Discord, et c'est elle qui a décidé de chaque détail — le pavillon épais, les trois disques par roue, le nez biseauté.

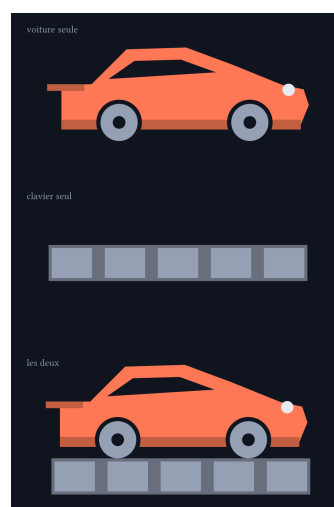


Figure 23. – Voiture et clavier partagent leur ligne de sol : la voiture roule littéralement sur les touches, sans aucune translation à écrire. C'est la métaphore du jeu — taper est ce qui la fait avancer.

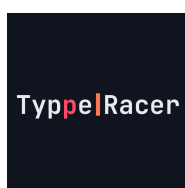


Figure 24. – Icône d'application.

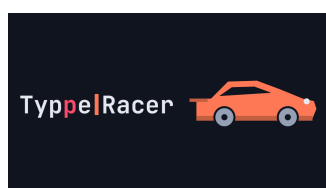


Figure 25. – Cover de l'étagère.



Figure 26. – Overlay de la grille.

Côté Rich Presence, le jeu pousse à Discord un état à chaque transition — menu, entraînement, salon, course — et Discord l'affiche aux autres membres du salon vocal. Le grand visuel change avec l'état, le

petit reste l'icône de l'application. Un détail a été tranché en cours de route : dans un salon d'attente, c'est le visuel du **Mode de jeu choisi** qui s'affiche, pas une image d'attente générique — ce qu'on est sur le point de jouer intéresse davantage le lecteur que le fait qu'on attende. L'image « salon » reste donc dans le dossier, sans emploi.



Figure 27. – menu



Figure 28. – practice



Figure 29. – race



Figure 30. – floor-is-lava



Figure 31. – spam



Figure 32. – lobby — produit, plus envoyé

### 5.11 Le graphe de résultats, sans bibliothèque

Le graphe seconde par seconde de l'écran de résultats reposait au départ sur une bibliothèque de graphiques. Elle a été retirée du projet et remplacée par un SVG construit à la main. Le raisonnement n'était pas la performance mais la proportion : le projet affiche **une** courbe, avec deux séries et une échelle fixée par le domaine. Le coût d'une dépendance qui sait tout dessiner — son poids dans le bundle, ses avis de sécurité à surveiller, sa version majeure à suivre — n'était pas payé par ce qu'on en utilisait.

### 5.12 La chaîne d'intégration et le déploiement

Trois travaux, dont deux tournent à chaque poussée et le troisième seulement sur la branche principale.

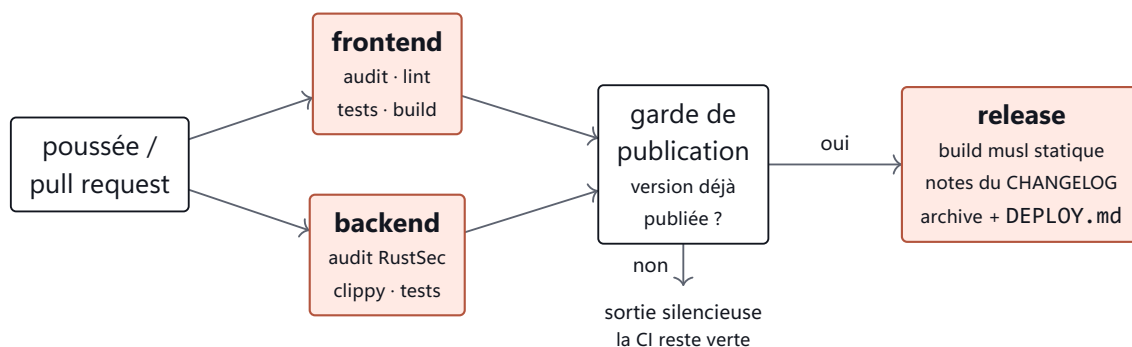


Figure 33. – La chaîne d'intégration. Le travail de publication **dépend** des deux autres, ce qui rend mécaniquement impossible de publier avec un test rouge — et c'est la raison pour laquelle il vit dans le même fichier plutôt que dans un workflow séparé.

La garde de publication lit la version du manifeste et la compare aux versions déjà publiées. Elle ne lit pas les étiquettes de version posées sur le dépôt : pousser une branche et pousser une étiquette sont deux événements sans ordre garanti entre eux, et une garde fondée là-dessus raterait une publication une fois sur deux, silencieusement.

La branche principale bouge pour deux raisons — de vraies versions, mais aussi des fusions de section et des poussées de documentation. Sur ces dernières, la version n'a pas changé : le travail sort alors sans rien faire et sans échouer. Le faire échouer en rouge apprendrait à ignorer la couleur de la CI.

#### RÉCIT Les gardes du travail de publication

**Problème** Le premier jet demandait à l'outil GitHub si la version était déjà publiée, **à l'intérieur d'une condition**. Deux pièges s'y superposaient. Le premier : dans une condition shell, l'arrêt automatique sur erreur est suspendu — une panne réseau, un quota ou un souci d'authentification se lisait donc « pas de release », et la publication partait. Le second : le titre de la version venait du fichier de notes et était interpolé directement dans la ligne de commande, où une paire de backquotes se serait **exécutée**. Ce n'était pas théorique : le fichier de notes du projet donne lui-même en exemple un titre contenant des backquotes.

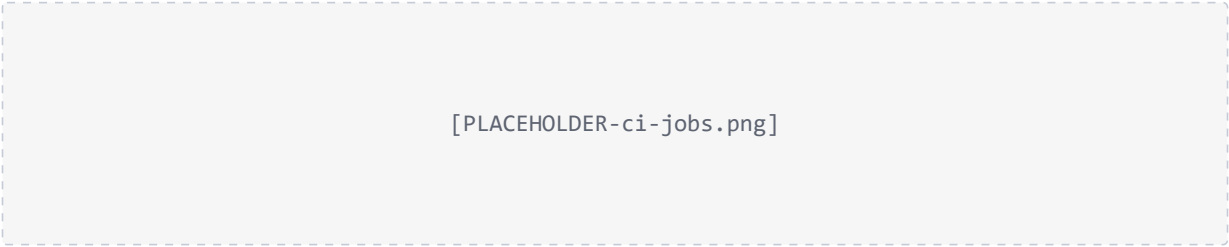
**Décision** Sortir l'appel réseau de la condition pour qu'un échec fasse honnêtement échouer le travail, lister les versions publiées plutôt que d'interroger l'une d'elles — un code de retour non nul ne distingue pas un « absent » d'une panne — et faire passer le titre par l'environnement, où aucun shell ne le relit.

**Résultat** Deux classes de bugs qui ne se seraient manifestées qu'en production : une publication de travers un jour de panne réseau, et une exécution de commande arbitraire déclenchée par un fichier de notes. Aucune des deux n'aurait été attrapée par un test.

#### EN CLAIR Un binaire « statique »

Un programme compilé va normalement chercher au démarrage des bibliothèques déjà installées sur la machine. Si celle-ci en a une version plus ancienne que la machine qui a compilé, il refuse de démarrer — et on ne s'en aperçoit qu'au déploiement. Un binaire statique emporte tout avec lui : c'est un seul fichier qu'on dépose sur n'importe quel Linux et qui démarre. Le projet peut se le permettre parce qu'il n'utilise aucune bibliothèque système — sa base de données est compilée dedans, et son chiffrement est écrit en Rust.

L'archive publiée contient le binaire, le build du frontend, et une notice de déploiement générée. Cette notice insiste sur un point : le serveur écoute **volontairement** sur la boucle locale et non sur le réseau. Il est injoignable depuis une autre machine, parce que le mode de déploiement du projet est un tunnel qui tourne sur le même hôte. Exposer directement sur Internet demande un proxy inverse devant, et c'est un choix à poser explicitement.



[PLACEHOLDER-ci-jobs.png]

Figure 34. – Les trois travaux verts sur une poussée. Le troisième n'apparaît que sur la branche principale.



[PLACEHOLDER-release.png]

Figure 35. – Une version publiée avec son archive. Les notes viennent du fichier de changements du dépôt, pas des titres de commits : une version parle au joueur.

### 5.13 Méthode : le modèle de domaine fait autorité sur le nommage

Le projet tient un glossaire de domaine à la racine, et ce glossaire n'est pas documentaire : il fait **autorité**. Chaque terme y porte non seulement sa définition mais la liste des mots interdits à sa place — « Race » interdit Match, Duel, Course ; « Abandon » interdit Quit, DNF. Le code, les issues et les messages d'interface s'y conforment.

L'intérêt s'est révélé sur les cas limites, pas sur les cas courants. Nommer « Brûlé », « Devancé » et « Échec » comme trois états **distincts** d'abandon — au lieu d'un seul état « perdu » à nuancer par un drapeau — a forcé à répondre à la question qui décide vraiment de leur comportement : **qui** a mis fin à ce Run ? Le joueur, sa propre faute, ou la comparaison aux autres ? Les cinq sorties du diagramme de la section 5.3 sont sorties de là, et le type qui les porte les rend exhaustives par construction : oublier d'en traiter une ne compile pas.

### 5.14 Méthode : la parité TypeScript ↔ Rust prouvée par vecteurs

L'algorithme de score existe deux fois : en TypeScript, où il est la **référence**, et en Rust, où il est autoritaire. Ce n'est pas une duplication accidentelle — le navigateur en a besoin pour le compteur en direct, le serveur en a besoin pour le verdict. Aucun générateur de types ne relie les deux : le miroir est **manuel**, et chaque fichier porte en en-tête le nom de son homologue.

#### RÉCIT Le miroir avait dérivé, sans filet

- Problème** Le décompte avant le départ était réglable, et le réglage semblait fonctionner : la valeur se choisissait, se rediffusait, s'affichait. Mais les deux côtés n'étaient plus d'accord sur la valeur par défaut — le décompte disait sept secondes d'un côté et cinq de l'autre. Cliquer sur le réglage ne faisait donc **rien** de visible, et rien ne signalait l'écart : ni erreur, ni test rouge, ni avertissement.
- Décision** Extraire des **vecteurs de test partagés** — des jeux d'entrée avec leur résultat attendu, dans un dossier à la racine — et faire lire ces mêmes fichiers par les deux suites de tests, celle de TypeScript et celle de Rust.
- Résultat** Une divergence entre les deux langages devient un test rouge dans l'un des deux, au lieu d'un comportement silencieusement faux. C'est le seul filet possible pour un miroir manuel — et le prix est modeste : les vecteurs sont des données, pas du code.

**EN CLAIR** Un « miroir manuel » entre deux langages

La même règle de calcul doit exister dans le navigateur et sur le serveur, qui ne parlent pas le même langage. On peut demander à un outil de traduire automatiquement l'un vers l'autre — c'est lourd, et l'outil devient une pièce du projet à entretenir. Ou on peut écrire les deux à la main et vérifier qu'ils disent la même chose, en leur donnant les mêmes questions et en comparant leurs réponses. Le projet a choisi la seconde option ; les « vecteurs » sont ces questions communes.

**5.15 Méthode : les décisions d'architecture consignées**

Dix-neuf décisions sont consignées, une par fichier, dans un dossier dédié. Chacune répond à une question qui ne se lit pas depuis le code seul : pourquoi une Race n'a pas de Mode, pourquoi le texte cible est persisté tel quel plutôt que régénéré depuis sa graine, pourquoi le message de fin porte les résultats et pas seulement l'ordre. La table complète est en annexe.

Leur valeur réelle est apparue tard. Quand un fichier de réseau a atteint trois mille trois cent soixante-et-onze lignes — le fil réseau, le salon et le moteur de course entassés au même endroit — la décision de le découper n'a pas eu à être redécouverte : les frontières étaient déjà nommées dans les décisions existantes, il ne restait qu'à les rendre visibles dans l'arborescence.

**RÉCIT** Un fichier de 3 371 lignes

**Problème** Tout le multijoueur vivait dans un seul fichier : la gestion du socket, l'état du salon, les réglages, et le moteur de course avec ses trois Modes de jeu. Ajouter un Mode de jeu revenait à toucher au même fichier que corriger une déconnexion — chaque chantier multijoueur entraînait en collision avec les autres.

**Décision** Découper selon les frontières que les décisions d'architecture nommaient déjà : le Mode de jeu comme table déclarée, les Réglages de salon avec un contrat de retour explicite, le moteur de course séparé du transport.

**Résultat** Le découpage n'a pas été une réécriture mais une **révélation** : les frontières existaient dans le vocabulaire du domaine avant d'exister dans les fichiers. C'est l'argument le plus concret que le projet ait produit en faveur des deux chantiers de méthode précédents (issue 205).

## 6 Documentation technique

---

*Fin de la première moitié. Les sections 6 à 8 sont ici en squelette : leurs titres fixent le plan et résolvent les renvois des sections précédentes, leur corps arrive au prochain lot.*

### 6.1 Arborescence du projet

À rédiger dans la seconde moitié — l'arbre racine au niveau dossier, puis `backend/src/` et `frontend/src/` détaillés fichier par fichier.

### 6.2 Architecture générale et frontière client/serveur

À rédiger dans la seconde moitié — diagrammes 1 (architecture globale, le backend a deux rôles) et 2 (carte des modules, avec la ligne du miroir manuel).

### 6.3 Le protocole WebSocket

À rédiger dans la seconde moitié — pourquoi WebSocket plutôt que du polling · les 29 messages en deux tableaux · la frontière de confiance · les trois portes d'entrée · le fan-out · les garde-fous · l'autorisation qui EST la connexion · le passage par `/.` proxy. Diagrammes 9 (une Race normale), 10 (Floor is lava).

### 6.4 Le contrat HTTP

À rédiger dans la seconde moitié — la table des endpoints — méthode, chemin, rôle, authentification. Aucun payload JSON : ils restent dans `Docs/API.md`.

### 6.5 Persistance

À rédiger dans la seconde moitié — 2 tables, 6 migrations, le record personnel **dérivé** sans table dédiée. Diagrammes 5 (schéma de la base) et 11 (soumission d'un Run solo).

### 6.6 Sécurité et frontière de confiance

À rédiger dans la seconde moitié — ce que le client a le droit d'affirmer, ce qu'il ne fait qu'**arrêter**, ce que le serveur possède seul.

### 6.7 Tests et assurance qualité

À rédiger dans la seconde moitié — 382 tests TypeScript, 159 tests Rust, les vecteurs partagés, ce que la CI garantit et ce qu'elle ne garantit pas.



## 7 Limites connues et suites

---

À rédiger dans la seconde moitié — les en-têtes de sécurité restants · le tunnel qui change d'URL à chaque redémarrage  
· l'absence de leaderboard · les Rooms qui meurent avec le processus · la dette assumée de PONYTAIL-DEBT.md.

## 8 Annexes

---

À rédiger dans la seconde moitié — la table des 19 décisions d'architecture · les documents légaux exigés par le portail Discord · les pointeurs vers `CONTEXT.md`, `Docs/API.md` et `Docs/PHASE2.md`.